

Q1 Multilevel Inheritance for Circular Economy

10 marks

A circular-economy system classifies products through multiple levels such as ElectronicDevice → Smartphone → RecyclableSmartphone. Implement the hierarchy.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class ElectronicDevice{
4     public:
5     void deviceType(){
6         cout<<"Electronic Device"<<endl;
7     }
8 };
9 class Smartphone:public ElectronicDevice{
10     public:
11     void smartphoneType(){
12         cout<<"Smartphone"<<endl;
13     }
14 };
15 class RecyclableSmartphone:public Smartphone{
16     public:
17     void recyclable(){
18         cout<<"Recyclable Smartphone"<<endl;
19     }
20 };
21 int main(){
22     RecyclableSmartphone r;
23     r.deviceType();
24     r.smartphoneType();
25     r.recyclable();
26     return 0;
27 }
```

Input

stdin input

Output

```
Electronic Device
Smartphone
Recyclable Smartphone
```

Visible Test Cases

Test Case 1

Q2 Composition in Solar-Powered Homes

10 marks

A solar-powered house contains a BatteryStorage component. Analyze the use of composition and evaluate its advantage over inheritance.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class BatteryStorage{
4     public:
5     void showBattery(){
6         cout<<"Battery Storage Component"<<endl;
7     }
8 };
9 class SolarPoweredHouse{
10     private:
11     BatteryStorage battery;
12     public:
13     void showHouse(){
14         cout<<"Solar Powered House"<<endl;
15         battery.showBattery();
16     }
17 };
18 int main(){
19     SolarPoweredHouse house;
20     house.showHouse();
21     return 0;
22 }
```



Run



Save

Input

stdin input

Output

```
Solar Powered House
Battery Storage Component
```

Visible Test Cases

Test Case 1

Q3 Array of Pointers for Smart Grids

10 marks

A smart-grid controller manages multiple energy sources dynamically. Analyze how arrays of pointers to objects improve scalability.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class EnergySource{
4     public:
5     virtual void display(){
6         cout<<"Energy Source"<<endl;
7     }
8 };
9 class Solar:public EnergySource{
10     public:
11     void display()override{
12         cout<<"Solar Energy"<<endl;
13     }
14 };
15 class Wind:public EnergySource{
16     public:
17     void display()override{
18         cout<<"Wind Energy"<<endl;
19     }
20 };
21 int main(){
22     Solar solar;
23     Wind wind;
24     EnergySource*sources[2];
25     sources[0]=&solar;
26     sources[1]=&wind;
27     for(int i=0;i<2;i++){
28         sources[i]->display();
29     }
30     return 0;
31 }
```

Input

stdin input

Output

```
Solar Energy
Wind Energy
```

Visible Test Cases

Test Case 1

Q4 Constructor Execution in Battery Recycling

10 marks

A battery-recycling application initializes common Battery information before creating LithiumIonBattery objects. Analyze the execution order of base and derived class constructors.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Battery{
4     public:
5     Battery(){
6         cout<<"Battery Constructor"<<endl;
7     }
8 };
9 class LithiumIonBattery:public Battery{
10     public:
11     LithiumIonBattery(){
12         cout<<"Lithium-Ion Battery Constructor"<<endl;
13     }
14 };
15 int main() {
16     LithiumIonBattery battery;
17     return 0;
18 }
```

Input

stdin input

Output

```
Battery Constructor
Lithium-Ion Battery Constructor
```

Visible Test Cases

Test Case 1

Q5 Virtual Base Class for Battery Data

10 marks

An EV battery-management system inherits common battery information through multiple classes, causing duplicate data. Analyze the issue and resolve it using a virtual base class.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Battery{
4     public:
5     int batteryID;
6     Battery(){
7         batteryID=101;
8     }
9     void showBattery(){
10         cout<<"Battery ID:"<<batteryID<<endl;
11     }
12 };
13 class ElectricVehicle:virtual public Battery{
14 };
15 class BatteryManagement:virtual public Battery{
16 };
17 class EVBatterySystem:public ElectricVehicle,public BatteryManagement{
18     public:
19     void display(){
20         showBattery();
21         cout<<"Virtual Base Class Used"<<endl;
22     }
23 };
24 int main(){
25     EVBatterySystem ev;
26     ev.display();
27     return 0;
28 }
```

Input

stdin input

Output

```
Battery ID:101
Virtual Base Class Used
```

Visible Test Cases

Test Case 1